

RadioSignals — CI/CD и Kubernetes деплојмент на повеќесервисна апликација

Мануел Трајчев, 221046

Линкови:

Git репозиториум: <https://github.com/ManuelTrajcev/RadioSignalsAnalysis>

Docker Hub: <https://hub.docker.com/r/manueltrajcev/radio-signals>

Содржина

1. Вовед и опис на апликацијата
2. Архитектура на системот
3. Git репозиториум
4. Докеризација на апликацијата
5. Оркестрација со Docker Compose
6. CI/CD pipeline со GitHub Actions
7. Kubernetes манифести
8. Демонстрација (посебен namespace + докажување дека работи)
9. Бонус: CD на cloud околина
10. Што е оригинално во однос на материјалите од предавања
11. Заклучок

1. Вовед и опис на апликацијата

RadioSignals е веб-апликација за анализа и предвидување на радиосигнали (мерења на FM и дигитални сигнали по населени места). Корисникот внесува мерења преку веб-интерфејс, а системот ги чува во база на податоци и повикува модел за машинско учење кој предвидува вредности (на пр. јачина на сигнал) врз основа на внесените параметри.

Апликацијата е составена од **четири сервиси**, секој спакуван во посебен Docker контејнер:

Сервис	Улога	Технологија	Порта	Изворен директориум
postgres	База на податоци	PostgreSQL 16	5432	Postgres/
ml	API за предвидување	Python 3.11 / FastAPI	8000	RadioSignalsML/
backend	REST API	.NET 10 / ASP.NET	8080	RadioSignalsBackEnd/
frontend	Веб-интерфејс	React + Vite + nginx	80	RadioSignalsFrontEnd/

Целта на проектот е целата апликација да биде:

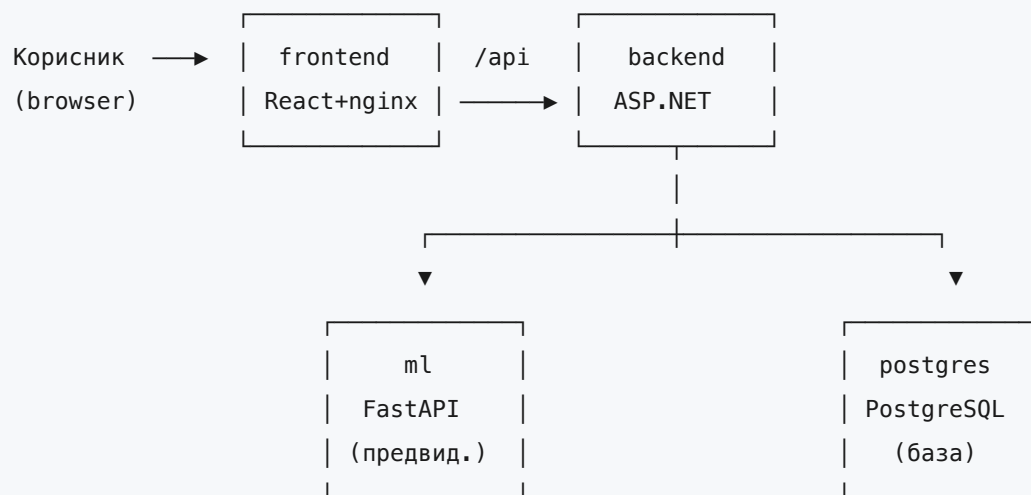
- ставена на јавен Git репозиториум,
- докеризирана (по еден `Dockerfile` за секој сервис),
- оркестрирана локално со Docker Compose,
- автоматски градена и објавувана преку CI/CD pipeline (GitHub Actions → Docker Hub),
- и распослана (deploy) на Kubernetes кластер во посебен namespace, со сите потребни објекти: Deployment, Service, Ingress, StatefulSet, ConfigMap и Secret.



Слика 1: Изглед на веб-интерфејсот на апликацијата (почетен екран / форма за внес на мерење).

2. Архитектура на системот

Текот на податоците низ системот е следен:



- **frontend** ги служи статичките React фајлови преку nginx и ги препраќа повиците кон `/api` до backend.
- **backend** ја содржи бизнис-логиката, комуницира со базата (`postgres`) преку Entity Framework Core и го повикува `ml` сервисот за предвидувања (`PredictionService__BaseUrl=http://ml:8000`).
- **ml** е посебен FastAPI сервис кој ги вчитува претренираните модели (`.joblib` артефакти) и враќа предвидувања.
- **postgres** ги чува сите перзистентни податоци.

Истите имиња на сервиси (`postgres`, `ml`, `backend`) се користат и во Docker Compose и во Kubernetes, така што конекциските стрингови и URL-овите остануваат непроменети меѓу двете околин.

3. Git репозиториум

Целиот изворен код, заедно со Dockerfile-овите, Docker Compose фајловите, CI/CD pipeline-от и Kubernetes манифестите, е јавно достапен на GitHub:

 <https://github.com/ManuelTrajcev/RadioSignalsAnalysis>

Структура на репозиториумот:

```
RadioSignalsAnalysis/
├── RadioSignalsBackEnd/      # .NET 10 REST API + unit тестови
├── RadioSignalsML/          # Python/FastAPI сервис за предвидување
├── RadioSignalsFrontEnd/    # React + Vite + nginx
├── Postgres/                # Custom PostgreSQL имиџ
├── compose.yml              # Docker Compose (локален build)
├── compose-hosted.yml       # Docker Compose (повлекување од Docker Hub)
├── k8s/                     # Kubernetes манифести (kustomize)
└── .github/workflows/
    └── ci-cd.yml            # CI/CD pipeline
```

Работата е организирана преку feature-гранки и Pull Request-и кон `main` гранката (видливо во историјата на commit-и и во merge-овите на репозиториумот), што е добра практика за тимска работа и code review.

 **Слика 2:** Почетна страница на GitHub репозиториумот со структурата на проектот.

4. Докеризација на апликацијата

Секој од четирите сервиси има свој `Dockerfile`, прилагоден на технологијата што ја користи.

4.1 Backend (.NET 10) — multi-stage build

```
# — Build stage —
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
COPY RadioSignals.sln ./
COPY Domain/Domain.csproj Domain/
COPY Repository/Repository.csproj Repository/
COPY Services/Services.csproj Services/
COPY RadioSignalsWeb/RadioSignalsWeb.csproj RadioSignalsWeb/
RUN dotnet restore RadioSignalsWeb/RadioSignalsWeb.csproj
COPY . .
RUN dotnet publish RadioSignalsWeb/RadioSignalsWeb.csproj -c Release -o /app/publish

# — Runtime stage —
FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS runtime
WORKDIR /app
COPY --from=build /app/publish .
COPY RadioSignalsWeb/SeedData ./SeedData
ENV ASPNETCORE_URLS=http://+:8080
EXPOSE 8080
ENTRYPOINT ["dotnet", "RadioSignalsWeb.dll"]
```

Оптимизации: проектните `.csproj` фајлови се копираат пред остатокот од кодот за да се искористи Docker layer caching при `restore` ; финалниот имиџ се базира на лесниот `aspnet runtime` наместо на целиот SDK.

4.2 ML сервис (Python / FastAPI)

```
FROM python:3.11-slim
# libgomp1 е потребна од scikit-learn (OpenMP) при инференца
RUN apt-get update && apt-get install -y --no-install-recommends libgomp1 \
    && rm -rf /var/lib/apt/lists/*
RUN adduser --disabled-password --gecos "" appuser
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY service/ ./service/
COPY artifacts/ ./artifacts/
USER appuser
ENV PREDICT_HOST=0.0.0.0 PREDICT_PORT=8000
EXPOSE 8000
CMD ["uvicorn", "service.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Безбедност: сервисот се извршува под непривилегиран корисник (`appuser`), а не како `root` . Зависностите се инсталираат во посебен слој (`cache`) пред копирањето на кодот.

4.3 Frontend (React + Vite + nginx) — multi-stage build

```
# — Build stage —
FROM node:22-alpine AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
ARG VITE_API_BASE_URL=http://localhost:5229/api
ENV VITE_API_BASE_URL=$VITE_API_BASE_URL
RUN npm run build

# — Runtime stage —
FROM nginx:stable-alpine AS runtime
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Базниот URL на API-то (`VITE_API_BASE_URL`) се „вградува“ во моментот на градење преку `build-argument`, што овозможува иста слика да се гради за различни околинис (локално, Kubernetes).

4.4 PostgreSQL (custom имиџ)

```
FROM postgres:16
ENV POSTGRES_USER=postgres
ENV POSTGRES_PASSWORD=postgres
ENV POSTGRES_DB=radiosignals
EXPOSE 5432
```

 **Слика 3:** Резултат од `docker build / docker images` со четирите изградени слики.

5. Оркестрација со Docker Compose

Подготвени се **два** Compose фајла за две различни намени:

5.1 `compose.yml` — локален развој (build од изворен код)

Ги гради сите слики локално од изворниот код. Содржи:

- **healthcheck** на `postgres` (`pg_isready`),
- **depends_on со услови** — `backend` чека `postgres` да биде „healthy“ и `ml` да стартува пред да се подигне,
- именуван volume `pgdata` за перзистенција на податоците,
- вградување на конекцискиот стринг и `PredictionService__BaseUrl` преку `environment` променливи.

```
backend:
  build:
    context: ./RadioSignalsBackEnd
    dockerfile: Dockerfile
  ports:
    - "5229:8080"
  environment:
    - ASPNETCORE_ENVIRONMENT=Development
    - ConnectionStrings__DefaultConnection=Host=postgres;Port=5432;Database=radiosignals;Usernam
    - PredictionService__BaseUrl=http://ml:8000
  depends_on:
    postgres:
      condition: service_healthy
    ml:
      condition: service_started
```

5.2 `compose-hosted.yml` — повлекување од Docker Hub

Наместо да гради, ги **повлекува** готовите слики објавени на Docker Hub (`manueltrajcev/radio-signals:api-1.0`, `:ml-1.0`, `:frontend-1.0`), што ја симулира продукциската ситуација каде имиците доаѓаат од регистар наместо да се градат на самата машина.

Стартување:


```
docker compose -f compose.yml up --build      # локален build
docker compose -f compose-hosted.yml up      # од Docker Hub
```

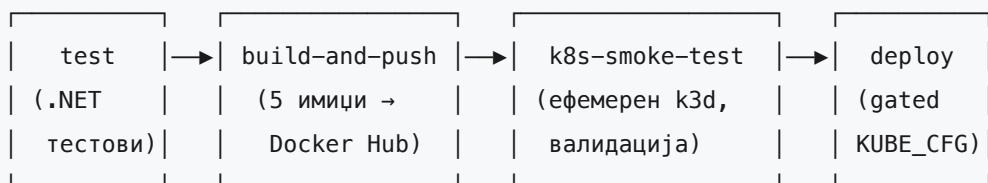
Апликацијата потоа е достапна на `http://localhost:3000` (frontend), кој комуницира со backend на `http://localhost:5229`.

 **Слика 4:** `docker compose up` со сите четири контејнери во „running“/„healthy“ состојба.

6. CI/CD pipeline со GitHub Actions

За CI платформа е избрана **GitHub Actions**. Pipeline-от е дефиниран во `.github/workflows/ci-cd.yml` и се состои од **четири фази** кои се извршуваат секвенцијално со зависности (`needs`):

```
test → build-and-push → k8s-smoke-test → deploy
```



6.1 Кога се извршува

```
on:
  push:
    branches: [main]
    paths: ['RadioSignalsBackEnd/**', 'RadioSignalsML/**',
            'RadioSignalsFrontEnd/**', 'Postgres/**', 'k8s/**',
            '.github/workflows/ci-cd.yml']
  pull_request:
    branches: [main]
  workflow_dispatch:
```

6.2 Фаза 1 — Тестови (CI)

Пред каков било build, се извршува backend unit-test суитата (xUnit). Така ниту една слика не се објавува ако тестовите паднат.

```
test:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-dotnet@v4
      with: { dotnet-version: '10.0.x' }
    - run: dotnet restore RadioSignals.sln
    - run: dotnet build RadioSignals.sln -c Release --no-restore
    - run: dotnet test RadioSignals.sln -c Release --no-build
```

6.3 Фаза 2 — Градење и објавување слики (CI → регистар)

Со **matrix стратегија** сите слики се градат и објавуваат паралелно. На секој push кон `main`, секоја слика добива:

- подвижен таг (`api-latest`, `ml-latest`, `db-latest`, `frontend-latest`, `frontend-k8s`),
- непроменлив таг со кратко git-SHA (`api-<sha>`, итн.) — за следливост и за детерминистички rollout.

```
build-and-push:
  needs: test                                # се објавува само ако тестовите се зелени
  if: github.event_name != 'pull_request'    # никогаш од PR
  strategy:
    matrix:
      include:
        - { name: backend,      tagprefix: api,      latesttag: api-latest }
        - { name: ml,          tagprefix: ml,        latesttag: ml-latest }
        - { name: frontend,    tagprefix: frontend,  latesttag: frontend-latest }
        - { name: frontend-k8s, tagprefix: frontend-k8s, latesttag: frontend-k8s }
        - { name: db,          tagprefix: db,        latesttag: db-latest }
  steps:
    - uses: docker/login-action@v3
      with:
        username: ${ secrets.DOCKERHUB_USERNAME }
        password: ${ secrets.DOCKERHUB_TOKEN }
    - uses: docker/build-push-action@v6
      with:
        push: true
        tags: ${ steps.meta.outputs.tags }
        cache-from: type=gha
        cache-to: type=gha,mode=max
```

Креденцијалите за Docker Hub се чуваат како **GitHub Secrets** (`DOCKERHUB_USERNAME`, `DOCKERHUB_TOKEN`) и никогаш не се изложени во кодот. Се користи и GitHub Actions cache (`type=gha`) за побрзо повторно градење.

Frontend сликата се гради во **две варијанти**: општа (`frontend-latest`) и Kubernetes-специфична (`frontend-k8s`) со релативен `VITE_API_BASE_URL=/api`, бидејќи во Kubernetes и frontend-от и backend-от се изложени преку истиот Ingress хост.

6.4 Фаза 3 — Smoke-test на ефемерен Kubernetes кластер

Оригинален дел од овој pipeline: пред кој било вистински deploy, се подига **привремен k3d кластер во самиот GitHub runner**, се применуваат манифестите, се чека сите работни

ресурси да станат „Ready“ и се проверува дека Ingress-от навистина го served frontend-от. Кластерот потоа се брише. Така, евентуална грешка во манифестите се фаќа **бесплатно** на секој push, без потреба од надворешен кластер.

```
k8s-smoke-test:
  needs: build-and-push
  steps:
    - run: curl -sfL https://raw.githubusercontent.com/k3d-io/k3d/main/install.sh | bash
    - run: k3d cluster create radio-signals-ci --port "8081:80@loadbalancer" --wait
    - run: kubectl apply -k k8s/
    - run: |
        kubectl -n radio-signals rollout status statefulset/postgres --timeout=180s
        kubectl -n radio-signals rollout status deploy/backend --timeout=300s
    - run: curl -fsS -H "Host: radiosignals.local" http://localhost:8081/ # Ingress
    - if: always()
      run: k3d cluster delete radio-signals-ci
```

6.5 Фаза 4 — CD (Vultr + k3s)

Последната фаза го применува манифестите на вистински кластер (Vultr + k3s) и го врши rollout-от на новоизградените слики. Во финалната верзија оваа фаза е **строга**: ако `KUBE_CONFIG` недостасува или API-то на кластерот не е достапно од GitHub runner, job-от паѓа со error (наместо да се прескокне), за да нема „зелено“ CI без реален deploy.

Во финалната верзија на workflow-от е додадена и поддршка за GitHub Actions variable `INGRESS_HOST`. Ако е поставена, deploy фазата автоматски го patch-ира `radiosignals-ingress` host-от (на пр. `149.28.123.110.sslip.io`) и нема потреба од рачно уредување на `k8s/ingress.yaml` за секој environment.

```

deploy:
  needs: k8s-smoke-test
  steps:
    - name: Check cluster reachability
      run: kubectl cluster-info --request-timeout=15s ...
    - name: Apply manifests & roll out git-SHA images
      run: |
        kubectl apply -k k8s/
        # optional host override from repository variable
        kubectl -n radio-signals patch ingress radiosignals-ingress ...
        kubectl -n radio-signals set image deploy/backend backend=$IMAGE_REPO:api-$SHA
        kubectl -n radio-signals set image deploy/frontend
frontend=$IMAGE_REPO:frontend-k8s-$SHA
        kubectl -n radio-signals set image deploy/ml ml=$IMAGE_REPO:ml-$SHA
        # verify pinning to immutable SHA tags
        kubectl -n radio-signals get deploy/frontend -o
jsonpath='{.spec.template.spec.containers[0].image}'
        kubectl -n radio-signals rollout status deploy/backend --timeout=300s

```

Потребни GitHub repository settings за CD:

- **KUBE_CONFIG** (Secret): kubeconfig за Vultr k3s кластерот со **server:**
`https://<PUBLIC_IP>:6443`
- **INGRESS_HOST** (Variable): јавен host за Ingress (во демонстрацијата:
`149.28.123.110.sslip.io`)

Забелешка: ако **INGRESS_HOST** не е поставен, Ingress останува со `radiosignals.local` и надворешниот пристап ќе враќа 404 поради Host mismatch.

 **Слика 5:** Успешно („зелено“) извршување на pipeline-от во GitHub Actions со сите четири фази.

 **Слика 6:** Таговите на сликите на Docker Hub (`api-*`, `ml-*`, `db-*`, `frontend-*`).

7. Kubernetes манифести

Сите манифести се сместени во директориумот `k8s/` и се организирани преку **kustomize** (`kustomization.yaml`), што овозможува целиот стек да се примени со една команда и сите објекти автоматски да паднат во истиот namespace.

```
# k8s/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: radio-signals
resources:
  - 00-namespace.yaml
  - postgres-secret.yaml
  - postgres-config.yaml
  - postgres-service.yaml
  - postgres-statefulset.yaml
  - ml-deployment.yaml
  - ml-service.yaml
  - backend-config.yaml
  - backend-deployment.yaml
  - backend-service.yaml
  - frontend-deployment.yaml
  - frontend-service.yaml
  - ingress.yaml
```

7.1 ConfigMaps и Secrets

Конфигурацијата е одвоена од сликите според 12-factor принципите:

Secret (чувствителни податоци — пристапни податоци за базата):

```
apiVersion: v1
kind: Secret
metadata:
  name: postgres-secret
type: Opaque
stringData:
  POSTGRES_USER: postgres
  POSTGRES_PASSWORD: postgres
  connection-string:
    "Host=postgres;Port=5432;Database=radio signals;Username=postgres;Password=postgres"
```

ConfigMaps (нечувствителна конфигурација):

```
# postgres-config.yaml
data:
  POSTGRES_DB: radiosignals
  PGDATA: /var/lib/postgresql/data/pgdata # подкатало → избегнува lost+found проблем
при initdb
---
# backend-config.yaml
data:
  ASPNETCORE_ENVIRONMENT: Production
  PredictionService__BaseUrl: "http://ml:8000"
```

⚠ За потребите на задачата се користат демо-креденцијали (postgres / postgres). Во продукција овие би доаѓале од менаџер на тајни (на пр. Sealed Secrets / External Secrets), а не би биле committed во репозиториумот.

7.2 Deployment за апликацијата

Трите апликациски сервиси (ml , backend , frontend) се распослани како Deployment објекти. Backend Deployment-от е најилустративен — ги покажува сите барани елементи: вчитување конфигурација од ConfigMap/Secret, init-контејнер, и health probes.

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
spec:
  replicas: 2
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0
      maxSurge: 1
  selector:
    matchLabels: { app: backend }
  template:
    metadata:
      labels: { app: backend }
    spec:
      initContainers:
        # Backend-от извршува EF Core миграции на старт; чекаме Postgres за да не влезе
        # во crash-loop.
        - name: wait-for-postgres
          image: busybox:1.36
          command: ["sh", "-c", "until nc -z postgres 5432; do echo waiting; sleep 2;
done"]
      containers:
        - name: backend
          image: manueltrajcev/radio-signals:api-latest
          ports:
            - containerPort: 8080
          envFrom:
            - configMapRef: { name: backend-config }      # нечувствителна конфигурација
          env:
            - name: ConnectionStrings__DefaultConnection
              valueFrom:
                secretKeyRef: { name: postgres-secret, key: connection-string } # од
Secret
          startupProbe:
            # дозволува ~150s за миграции + seeding
            tcpSocket: { port: 8080 }
            periodSeconds: 5
            failureThreshold: 30
          readinessProbe:
            tcpSocket: { port: 8080 }
          livenessProbe:

```



```

    tcpSocket: { port: 8080 }
    initialDelaySeconds: 30
  resources:
    requests: { cpu: "100m", memory: "256Mi" }
    limits:   { cpu: "500m", memory: "512Mi" }

```

Клучни моменти:

- **init-контејнер** `wait-for-postgres` спречува `crash-loop` додека базата не стане достапна,
- **startupProbe** остава доволно време за EF Core миграциите и seeding-от при првото подигнување,
- конфигурацијата се внесува преку `envFrom` (`ConfigMap`) и `valueFrom / secretKeyRef` (`Secret`),
- дефинирани се **resource requests/limits**.

`ml` и `frontend` Deployment-ите следат иста шема, со HTTP health probes (`/health` за `ml`, `/` за `frontend`). Во финалната конфигурација и трите апликациски Deployment-и (`backend`, `frontend`, `ml`) се поставени со `replicas: 2` и rolling update стратегија (`maxUnavailable: 0`, `maxSurge: 1`) за да се минимизира downtime при deploy.

7.3 Service за апликацијата

За секој работен ресурс е дефиниран `Service` за да биде достапен преку DNS-име во namespace-от:

```

# ml-service.yaml
apiVersion: v1
kind: Service
metadata: { name: ml }
spec:
  selector: { app: ml }
  ports:
    - port: 8000
      targetPort: 8000

```

Service	Тип	Порта	Намена
<code>ml</code>	ClusterIP	8000	backend го повикува на <code>http://ml:8000</code>
<code>backend</code>	ClusterIP	8080	Ingress + frontend го повикуваат
<code>frontend</code>	ClusterIP	80	Ingress рутира <code>/</code> тука
<code>postgres</code>	Headless (None)	5432	стабилен DNS за StatefulSet

Имињата на сервисите се идентични со оние во Docker Compose, така што конекциските стрингови работат непроменети.

7.4 Ingress за апликацијата

Целата апликација е изложена преку еден хост, со рутирање по патека — `/api` оди кон backend, сè друго кон frontend. Се користи **Traefik** ingress контролерот (стандарден во k3s/k3d).

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: radiosignals-ingress
spec:
  ingressClassName: traefik
  rules:
    - host: radiosignals.local
      http:
        paths:
          - path: /api          # подолгиот префикс има приоритет (Traefik)
            pathType: Prefix
            backend:
              service: { name: backend, port: { number: 8080 } }
          - path: /
            pathType: Prefix
            backend:
              service: { name: frontend, port: { number: 80 } }
```

Во локална околина host-от е `radiosignals.local`, а во cloud deploy workflow-от автоматски го заменува со вредноста од `INGRESS_HOST` (во демонстрацијата: `149.28.123.110.sslip.io`). Бидејќи и frontend-от и API-то се на истиот хост, нема проблеми со CORS — прелистувачот ги праќа сите повици кон истиот origin.

7.5 StatefulSet за базата со ConfigMaps/Secrets

Базата е распослана како **StatefulSet** (а не Deployment), бидејќи бара стабилен мрежен идентитет и перзистентен стораж преку `volumeClaimTemplates`.

```

apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
spec:
  serviceName: postgres          # headless Service-от од 7.3
  replicas: 1
  selector:
    matchLabels: { app: postgres }
  template:
    metadata:
      labels: { app: postgres }
    spec:
      containers:
        - name: postgres
          image: manueltrajcev/radio-signals:db-latest
          ports:
            - containerPort: 5432
          envFrom:
            - configMapRef: { name: postgres-config }    # POSTGRES_DB, PGDATA
            - secretRef:   { name: postgres-secret }     # POSTGRES_USER,
POSTGRES_PASSWORD
          volumeMounts:
            - name: pgdata
              mountPath: /var/lib/postgresql/data
          readinessProbe:
            exec: { command: ["pg_isready", "-U", "postgres", "-d", "radiosignals"] }
          livenessProbe:
            exec: { command: ["pg_isready", "-U", "postgres", "-d", "radiosignals"] }
      volumeClaimTemplates:
        - metadata: { name: pgdata }
          spec:
            accessModes: ["ReadWriteOnce"]
            storageClassName: local-path          # стандарден storage class во k3s
            resources:
              requests: { storage: 2Gi }

```

`volumeClaimTemplates` автоматски создава `PersistentVolumeClaim` (2Gi), така што податоците преживуваат рестарт/повторно распоредување на pod-от. Конфигурацијата и тајните се внесуваат преку `envFrom` (`configMapRef` + `secretRef`).

 **Слика 7:** Излез од `kubectl get statefulset,pvc -n radio-signals`.

8. Демонстрација (посебен namespace + докажување дека работи)

Сите објекти се распослани во посебен namespace `radio-signals` (дефиниран преку `00-namespace.yaml` и наметнат од `kustomization.yaml`), на локален `k3d` кластер.

8.1 Распослање

```
# Подигнување на локален k3d кластер (еднократно)
k3d cluster create mycluster --port "80:80@loadbalancer"

# Распослање на целиот стек со една команда
kubectl apply -k k8s/

# Мапирање на хостот (еднократно)
echo "127.0.0.1 radiosignals.local" | sudo tee -a /etc/hosts
```

8.2 Докажување дека работи

```
kubectl get all -n radio-signals
kubectl get ingress,configmap,secret,pvc -n radio-signals
```

Очекуван резултат:

- сите pod-ови (`postgres-0`, `ml`, `backend`, `frontend`) во состојба `Running` / `Ready`,
- активни Service-и, Ingress, ConfigMaps, Secrets и PVC,
- веб-интерфејсот достапен на <http://radiosignals.local>,
- внес на ново мерење / предвидување потврдува дека целиот тек `frontend` → `backend` → `ml` + `postgres` функционира.

8.3 Доказ за перзистенција

```
kubectl delete pod postgres-0 -n radio-signals    # се брише pod-от на базата
# StatefulSet-от го рекреира pod-от; претходно внесените податоци сè уште постојат
```

 **Слика 8:** `kubectl get all -n radio-signals` со сите ресурси во „Running“. 

Слика 9: Апликацијата отворена на `http://radiosignals.local` со успешно предвидување.  **Слика 10:** Доказ за перзистенција — податоците опстануваат по бришење на `postgres-0`.

8.4 Cloud демонстрација (Vultr + k3s)

Покрај локалната демонстрација, направена е и cloud демонстрација на Vultr инстанца со k3s, каде pipeline-от успешно извршува CD.

Чекори што се употребени:

1. На Vultr се инсталира k3s и се отвораат портите 80/443/6443 .
2. Од `k3s.yaml` се креира kubeconfig со јавна IP:

```
PUBLIC_IP=149.28.123.110
sudo sed "s#127.0.0.1#{PUBLIC_IP}#g" /etc/rancher/k3s/k3s.yaml > ~/kubeconfig-
vultr.yaml
```

3. kubeconfig-от се додава како GitHub Secret `KUBE_CONFIG` .
4. Се додава GitHub Variable `INGRESS_HOST=149.28.123.110.sslip.io` .
5. По push на `main` , deploy job-от ги применува манифестите, го patch-ира Ingress host-от и ги rollout-ира новите SHA тагови.

Јавен пристап за cloud демонстрацијата:

- <http://149.28.123.110.sslip.io>

9. Бонус: CD на cloud околина

Pipeline-от содржи `deploy` фаза за вистински кластер и е реализиран на **Vultr cloud инстанца** (Ubuntu + k3s):

1. Се подига Ubuntu инстанца со јавна IP адреса.
2. Се инсталира k3s и се проверува достапноста на API endpoint-от.
3. Се отвораат портите 6443/80/443 (Vultr Firewall + UFW на машината).
4. kubeconfig-от (со `server: https://<PUBLIC_IP>:6443`) се чува како GitHub Secret `KUBE_CONFIG` .
5. Се додава `INGRESS_HOST` variable (sslip.io host), за автоматски patch на Ingress host при deploy.
6. На секој push кон `main` , workflow-от автоматски ги rollout-ира новите `api-<sha>` , `ml-<sha>` , `frontend-k8s-<sha>` и `db-<sha>` слики.
7. Deploy job-от верифицира дека workload-ите навистина се пинувани на SHA тагови (а не на floating `*-latest` / `frontend-k8s`).

По ова, секој push кон `main` автоматски ја распослува новата верзија на cloud кластерот (целосно CI/CD).

10. Што е оригинално во однос на материјалите од предавања

- **Matrix-базиран** build на 4 (5 со `frontend-k8s`) сервиси во една задача, наместо посебни задачи по сервис.
- Двојна шема на тагирање: подвижен таг + непроменлив `git-SHA` таг за детерминистички rollout.
- **Ефемерен k3d smoke-test во CI** — оригинална додадена фаза која ги валидира манифестите на секој push без надворешен кластер.
- `deploy` фаза со строга проверка на достапност (fail-fast ако кластерот не е reachable) и верификација на SHA image pinning.
- Custom имиња (`radio-signals namespace`, `radiosignals.local` за локално + `INGRESS_HOST override` за cloud, специфични имиња на сервиси) и организација преку `kustomize`.
- `init`-контејнер + `startupProbe` прилагодени за EF Core миграциите на backend-от.

11. Заклучок

Со овој проект апликацијата RadioSignals е целосно автоматизирана: од push на код, преку автоматско тестирање, градење и објавување на Docker слики, до валидација на Kubernetes манифестите и распослање во посебен namespace. Сите барани компоненти — Dockerfile-ови, Docker Compose, CI/CD pipeline, Deployment, Service, Ingress, StatefulSet, ConfigMaps и Secrets — се имплементирани и демонстрирани, а pipeline-от е подготвен и за целосно CD на cloud околина.

Сумарна табела на исполнетост

#	Барање	%	Статус
1	Апликација на јавен git репозиториум	10	✓ GitHub
2	Докеризација	10	✓ 4 Dockerfile-ови
3	Оркестрација со Docker Compose	10	✓ <code>compose.yml</code> + <code>compose-hosted.yml</code>
4	CI платформа + pipeline → регистар (+ bonus CD)	20	✓ GitHub Actions → Docker Hub (+ deploy фаза)
5	Deployment + ConfigMaps/Secrets	10	✓ <code>ml/backend/frontend</code>
6	Service	10	✓ 4 Service-и
7	Ingress	10	✓ Traefik, host-base рутирање
8	StatefulSet за база + ConfigMaps/Secrets	10	✓ postgres + PVC
9	Посебен namespace + демонстрација	10	✓ <code>radio-signals</code> на k3d

Прилози / Линкови

- **Изворен код:** <https://github.com/ManuelTrajcev/RadioSignalsAnalysis>
- **Docker имиџи:** <https://hub.docker.com/r/manueltrajcev/radio-signals>
- **CI/CD pipeline:** `.github/workflows/ci-cd.yml`
- **Kubernetes манифести:** `k8s/`